# AsmetaComp: a Tool for Runtime Contract Checking with I/O Abstract State Machines *Tool Artifact Description* ⋆

Silvia Bonfanti[1][0000−0001−9679−4551], Angelo Gargantini[1][0000−0002−4035−0131], Elvinia Riccobene[2][0000−0002−1400−1026], and Patrizia Scandurra[1][0000−0002−9209−3624]

[1] University of Bergamo, Bergamo, Italy
{silvia.bonfanti,angelo.gargantini,patrizia.scandurra}@unibg.it
[2] Università degli Studi di Milano, Milano, Italy elvinia.riccobene@unimi.it

**Abstract.** Contract-based Design specifies components via *assume–guarantee* contracts, supporting modular development and verification of complex systems. In state-based modeling, it enables decomposition into interacting components coordinated through I/O events.
This artifact provides AsmetaComp, a tool for the compositional execution of systems specified using Input/Output Abstract State Machines (ASMs), enriched with assume–guarantee contracts. The tool supports the specification of component behavior and contracts in a unified environment, the definition of composition scripts, and the automated checking of contracts during execution.
The artifact includes the tool, a healthcare case study, and a set of executable scripts that allow users to reproduce different validation scenarios and observe both correct executions and contract violations.

**Keywords:** Contract-based Design · Models composition · Runtime Contract Checking · Abstract State Machines

## 1 Running case study

We illustrate the usage of AsmetaComp through a system example in the medical domain, modeled as a composition of I/O ASMs. Inspired by [2,1], the case study (shown in Fig.1) is about a *Medicine Reminder and Monitoring (MRM) System* designed to control a smart medicine dispenser (a remotely controllable pillbox) for medicine administration. The (client-side) system consists of two main sub-systems:

- *Electronic pill reminder and dispenser* (pillbox) with several compartments and slots per compartment;
- *Remote control mobile app* running on a smartphone, which monitors and configures the pillbox via local wireless (Bluetooth or Wi-Fi Direct).

⋆ Artefacts available in https://doi.org/10.5281/zenodo.18678488.
This artifact refers to the paper "AsmetaComp*: a Tool for Runtime Contract Checking with I/O Abstract State Machines*"

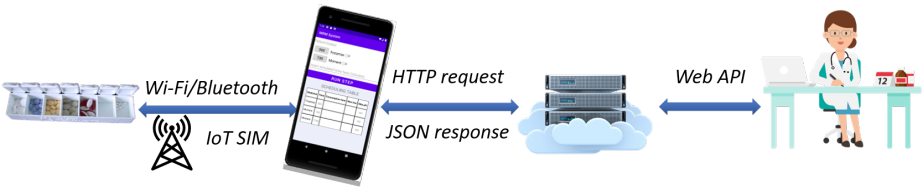


**Fig. 1.** MRM system: high-level architecture

A cloud-based information system provides persistent services for health records, prescriptions, and treatments entered by doctors through a web application. In this architecture, the smartphone acts as a *gateway* between the pillbox and the cloud, to synchronize patient data with the cloud using the smartphone's Internet connection (Wi-Fi or cellular). Our running example focuses only on the patient side system. The mobile app is responsible for checking the pillbox configuration after it is filled by the caregiver and connected to power, and for dynamically rescheduling pills to ensure safe drug intake. Unlike typical systems that only notify caregivers of missed doses, this solution automatically reschedules missed doses while ensuring no harmful drug interferences occur.

## 2 ASMETA models

This section provides the complete ASMETA models of the *ConfigurationMgr* (see Listing 1) and the *Rescheduler* (see Listing 2) to help better understand the MRM System.

```
asm configurationMgr
import StandardLibrary
import knowledge
export *

signature:
 enum domain States = {INIT | NORMAL}
 controlled state: States
 controlled prevRedLed: Compartment -> LedLights
 controlled nextDrugIndexN: Compartment ->Natural
 //INPUT from prescription
 monitored amount: String -> Natural
 monitored minTimeToIntake: String -> Natural
 monitored minToInterferer: Prod(String,String) -> Natural
 monitored time: String -> Seq(Natural)
 monitored medicine_list: Seq(String)
 monitored deltaDelay: String -> Natural
 //IN from Pillbox
 monitored time_consumption: Compartment -> Seq(Natural)
 monitored name: Compartment -> String
 monitored drugIndex: Compartment -> Natural
 monitored redLed: Compartment -> LedLights

 derived ledStatusUpdateOk: Compartment -> Boolean

 static compartment1: Compartment
 static compartment2: Compartment

definitions:

 function minTimeToIntake($medicine in Medicine) = minTimeToIntake(id($medicine))
```

```
function minToInterferer($m in Medicine, $n in Medicine) = minToInterferer(id($m),id($n))
function time($m in Medicine) = time(id($m))
function deltaDelay($m in Medicine) = deltaDelay(id($m))
function nextDrugIndex($compartment in Compartment) =  let ($i = drugIndex($compartment) + 1n) in
 if $i < iton(length(time_consumption($compartment))) then $i else 0n endif endlet
function ledStatusUpdateOk($compartment in Compartment) = ((prevRedLed($compartment) = OFF and
        redLed($compartment)= BLINKING) or (prevRedLed($compartment) = BLINKING and
        redLed($compartment)= ON))
function next($compartment in Compartment) = {$c in Compartment |
        (at(time_consumption($c),drugIndex($c)) >
        at(time_consumption($compartment),drugIndex($compartment))) : $c}

rule r_keepPrevLiths =
forall $compartment in Compartment do
 par
  //Keep previous led status
  prevRedLed($compartment) := redLed($compartment)
  nextDrugIndexN($compartment):=nextDrugIndex($compartment)
 endpar
rule r_INIT =
 par
  forall $s in asSet(medicine_list) do
   extend Medicine with $m do
     id($m) := $s
  state := NORMAL
 endpar
rule r_NORMAL_FUNCT = r_keepPrevLiths[]
   //**********ASSERTIONS DEFINITION**********
   //The only valid traces of a led light are the sequences: OFF, ON, BLINKING and BLINKING, OFF
   invariant inv_A_redLed over redLed: ( forall $c in Compartment with not ledStatusUpdateOk($c) )
   //Knowledge consistency: pills type consistency between the prescription and the compartments of the
         pillbox
   invariant inv_A_medicineList over name(Compartment): ( forall $c in Compartment with (
         contains(medicine_list,name($c))))
   invariant inv_A_compName over Compartment: (forall $m in asSet(medicine_list) with (exist $c in
         Compartment with name($c)=$m))
   //Pills amount consistency: the amount of pills to intake and the number of the corresponding
         compartment slots must be the same
   invariant inv_A_CompSize over Compartment: ( forall $c in Compartment with
         iton(length(time_consumption($c))) = amount(name($c)) )
   //Pills time consistency: the time schedule of the compartment's slots must be the same of those
         scheduled for the medicine and there must be no interferences
    invariant inv_A_CompMed over Compartment, Medicine: state=INIT implies (forall $c in
          Compartment with (((drugIndex($c) < iton(length(time_consumption($c))) and
    at(time_consumption($c),drugIndex($c)) = (at(time(name($c)),drugIndex($c)))))))
   /*The difference between its intake time and the intake time of the subsequent medications taken (in the
         next slot of the same compartment or in another compartment) must be greater than or equal to
         the medication's minToInterferer value with the subsequent medications */
  invariant inv_A_timeConsumption over time_consumption(Compartment): (forall $compartment in
        Compartment with (forall $c in next($compartment) with
        ((at(time_consumption($c),drugIndex($c))!= 0n and
        at(time_consumption($compartment),drugIndex($compartment))!= 0n ) implies
        ((iton((at(time_consumption($c),drugIndex($c)) −
        at(time_consumption($compartment),drugIndex($compartment)))) >=
        (minToInterferer(name($compartment),name($c))) and $c!=$compartment) or
        (iton((at(time_consumption($c),nextDrugIndex($c)) −
        at(time_consumption($compartment),drugIndex($compartment)))) >=
        (minToInterferer(name($compartment),name($c))) and $c=$compartment)))))

main rule r_Main =
 if state = INIT then
  r_INIT[] //Medicine knowledge initialization depending on how the pillbox has been filled
 else
  r_NORMAL_FUNCT[]
 endif

default init s0:
function state = INIT
```

```
    function medicine_list = ["fosamax","moment"]
    function amount($medicine in String) =
  switch($medicine)
   case "moment" : 2n
   case "fosamax" : 1n
  endswitch

 function time($m in String) =
  switch($m)
   case "moment" : [730n, 1140n]
   case "fosamax" : [360n]
  endswitch

 function minTimeToIntake($medicine in String) =
  switch($medicine)
   case "moment" : 120n
   case "fosamax" : 360n
   otherwise 0n
  endswitch

 function minToInterferer($m in String, $n in String) =
   if ($m = $n) then
    minTimeToIntake($m)
  else
  switch ($m,$n)////(($m,$n))
   case ("fosamax","moment") : 360n
   case ("moment","fosamax") : 360n
   otherwise 0n
  endswitch
  endif
```

**Listing 1.** ASMETA model of the ConfigurationMgr

```
asm rescheduler
import StandardLibrary
import knowledge
signature:
 enum domain States = {INIT | NORMAL}

 //IN from Pillbox
 monitored isPillMissed: Compartment -> Boolean //pill missed
 monitored pillTakenWithDelay: Compartment -> Boolean // is true if the patient takes the pill but it
      causes mintointerfer violation
 monitored actual_time_consumption: Compartment -> Seq(Natural) //Which time is the pill taken
 monitored day: Integer //Day number since first day of use
 monitored time_consumption: Compartment -> Seq(Natural)
 monitored drugIndex: Compartment -> Natural
 monitored name: Compartment -> String
 monitored redLed: Compartment -> LedLights

 //OUT to Pillbox
 out setNewTime: Compartment -> Boolean //if true set new time when pill is missed
 out setOriginalTime: Compartment -> Boolean //pill is missed and new time must be set = previous +
      delay, if mintointerfer is violated, this is false and the time is set to the original value
 out newTime: Compartment -> Natural //new pill time
 out skipNextPill: Prod(Compartment,Compartment) -> Boolean //is next compartment skipped? It can be
      that more than one next compartments are skipped (current compartment, skip compartment)
 out skipNextPill: Compartment -> Boolean //SafePillbox detected that pill taken with delay

 //INPUT from prescription
 monitored amount: String -> Natural
 monitored minTimeToIntake: String -> Natural
 monitored minToInterferer: Prod(String,String) -> Natural
 monitored time: String -> Seq(Natural)
 monitored medicine_list: Seq(String)
 monitored deltaDelay: String -> Natural
```

```
controlled state: States
controlled nextDrugIndexN: Compartment ->Natural
controlled tempComputation: Boolean
derived computeDer: Boolean
static compartment1: Compartment
static compartment2: Compartment
monitored pillboxSystemTime: Natural

definitions:

function computeDer = (forall $compartment in Compartment with
 ((forall $c in next($compartment) with  ( isPillMissed($compartment) and
        ((iton((at(time_consumption($c),drugIndex($c)) -
        at(time_consumption($compartment),drugIndex($compartment))-ntoi(deltaDelay(name($compartment)))))>=
        (minToInterferer(name($compartment),name($c))) and $c!=$compartment) or
        (iton((at(time_consumption($c),nextDrugIndex($c)) -
        at(time_consumption($compartment),drugIndex($compartment))-ntoi(deltaDelay(name($compartment)))))
        >= (minToInterferer(name($compartment),name($c))) and $c=$compartment)))) implies
        ((newTime ($compartment) =
        at(time_consumption($compartment),drugIndex($compartment))+deltaDelay(name($compartment))))))
function next($compartment in Compartment) = {$c in Compartment |
       (at(time_consumption($c),drugIndex($c)) >
       at(time_consumption($compartment),drugIndex($compartment))) : $c}
function minTimeToIntake($medicine in Medicine) = minTimeToIntake(id($medicine))
   function minToInterferer($m in Medicine, $n in Medicine) = minToInterferer(id($m),id($n))
   function time($m in Medicine) = time(id($m))
   function deltaDelay($m in Medicine) = deltaDelay(id($m))
   function nextDrugIndex($compartment in Compartment) =  let ($i = drugIndex($compartment) +
          1n) in if $i < iton(length(time_consumption($compartment))) then $i else 0n endif endlet

rule r_rescheduler =
  forall $compartment in Compartment do
  par
   if isPillMissed($compartment) then //M
    par
     setNewTime ($compartment):= true
   //Missed pill check if the new time causes invariant violation
    if ((forall $c in next($compartment) with ( (iton((at(time_consumption($c),drugIndex($c)) -
          at(time_consumption($compartment),drugIndex($compartment))-ntoi(deltaDelay(name($compartment)))))
          >= (minToInterferer(name($compartment),name($c))) and $c!=$compartment) or
          (iton((at(time_consumption($c),nextDrugIndex($c)) -
          at(time_consumption($compartment),drugIndex($compartment))-ntoi(deltaDelay(name($compartment)))))
          >= (minToInterferer(name($compartment),name($c))) and $c=$compartment)))) then //A
        par
         setOriginalTime ($compartment):= false
         newTime ($compartment):=
                at(time_consumption($compartment),drugIndex($compartment))+deltaDelay(name($compartment))
        endpar // if not update new time of pill otherwise skip the pill
    else
       par
        setOriginalTime ($compartment):= true
        newTime ($compartment):= at(time(name($compartment)),drugIndex($compartment))
       endpar
    endif
  endpar
 else
  setNewTime ($compartment):= false
 endif
  if pillTakenWithDelay($compartment) then
  //pill taken later compared the timecompartment: for all next pills that cause invariant violation because
        the current has been taken after set time -> skip pill
  forall $c2 in next($compartment) do
     if (((iton(at(time_consumption($c2),drugIndex($c2)) - (pillboxSystemTime mod 1440n)) <=
          (minToInterferer(name($compartment),name($c2)))) and $c2!=$compartment) or
          ((iton(at(time_consumption($c2),nextDrugIndex($c2)) - (pillboxSystemTime mod
          1440n))<=(minToInterferer(name($compartment),name($c2)))) and $c2=$compartment)) then
    par
    skipNextPill($compartment, $c2):= true
```

```
        skipNextPill($compartment) := true
       endpar
      endif
    else
     par
      forall $c3 in next($compartment) do
       skipNextPill($compartment, $c3):= false
      skipNextPill($compartment) := false
     endpar
    endif
   endpar

  rule r_resetMidnight =
    if (rtoi(pillboxSystemTime/1440n))> day then
     forall $c in Compartment do
      par
       setNewTime($c) := false
       skipNextPill($c) := false
       forall $c2 in Compartment with $c!=$c2 do
        skipNextPill ($c, $c2) := false
      endpar
  endif

   rule r_evaluate_output_pill=
     forall $compartment in Compartment do
      forall $c2 in next($compartment) do
       if skipNextPill($compartment) and setNewTime($compartment) and skipNextPill($compartment, $c2)
            then skip endif

   rule r_NORMAL_FUNCT =
        par r_rescheduler[] r_resetMidnight[] r_evaluate_output_pill[] endpar

  //If pill is missed and there is no overlap with next pills, delayed it
  invariant inv_G_newTime over newTime: (forall $compartment in Compartment with ((forall $c in
        next($compartment) with (isPillMissed($compartment) and
        ((iton((at(time_consumption($c),drugIndex($c))
        −at(time_consumption($compartment),drugIndex($compartment))−ntoi(deltaDelay(name($compartment)))))
        >= (minToInterferer(name($compartment),name($c))) and $c!=$compartment) or
        (iton((at(time_consumption($c),nextDrugIndex($c)) −
        at(time_consumption($compartment),drugIndex($compartment))−ntoi(deltaDelay(name($compartment)))))
        >= (minToInterferer(name($compartment),name($c))) and $c=$compartment))) implies ((newTime
        ($compartment) =
        at(time_consumption($compartment),drugIndex($compartment))+deltaDelay(name($compartment)))))))

  //If pill overlaps with next pills, skip the next pill that overlaps
  invariant inv_G_skipNextPill over skipNextPill(Compartment): (forall $compartment in Compartment
        with ((forall $c in next($compartment) with   (pillTakenWithDelay($compartment) and
        ((iton(at(time_consumption($c),drugIndex($c)) − (pillboxSystemTime mod 1440n))  <=
        (minToInterferer(name($compartment),name($c)))) and $c!=$compartment) or
        ((iton(at(time_consumption($c),nextDrugIndex($c)) − (pillboxSystemTime mod 1440n))<=
        (minToInterferer(name($compartment),name($c)))) and $c=$compartment)) implies
        (skipNextPill($compartment, $c)= true and skipNextPill($compartment)= true))))

  /*For a medication in a given compartment and slot that has been taken, the difference between its ACTUAL
        intake time and the intake time of the subsequent medications taken (in the next slot of the same
        compartment or in another compartment) must be greater than or equal to the medication's
        minToInterferer value with the subsequent medications */
  invariant inv_A_actual_time over Compartment: (forall $compartment in Compartment with (forall $c
        in next($compartment) with ((at(actual_time_consumption($c),drugIndex($c))!= 0n and
        at(actual_time_consumption($compartment),drugIndex($compartment))!= 0n ) implies (
        (iton((at(actual_time_consumption($c),drugIndex($c)) −
        at(actual_time_consumption($compartment),drugIndex($compartment)))) >=
        (minToInterferer(name($compartment),name($c))) and $c!=$compartment) or
        (iton((at(actual_time_consumption($c),nextDrugIndex($c)) −
        at(actual_time_consumption($compartment),drugIndex($compartment)))) >=
        (minToInterferer(name($compartment),name($c))) and $c=$compartment)))))

   main rule r_Main =
```

```
120     if state=INIT then
121     par
122      forall $c in Compartment do
123        par
124          skipNextPill($c):=false
125          setNewTime($c):=false
126        endpar
127      state:=NORMAL
128     endpar
129     else
130     par
131      tempComputation:=computeDer
132      r_NORMAL_FUNCT[]
133     endpar
134    endif

136   default init s0:
137    function state = INIT
138    function setNewTime($c in Compartment) = false
139    function newTime($c in Compartment) = 0n
140    function skipNextPill($c in Compartment, $c2 in Compartment) = false
141    function skipNextPill($c in Compartment) = false

143    //Initialization of day
144    function day = 0
145    // Initialization of the SystemTime
146    function pillboxSystemTime = 0n

148    // Turn—off all the led of the Compartments
149    function redLed($compartment in Compartment) = OFF
150    // Every compartment has an index starting from 0
151    function drugIndex($compartment in Compartment) = 0n

153    // Initialization of the actual time consumption for a compartment
154    function actual_time_consumption($compartment in Compartment) =
155     switch($compartment)
156      case compartment1 : [0n]
157      case compartment2 : [0n, 0n]
158     endswitch
```

**Listing 2.** ASMETA model of the Rescheduler

## 3 Tool Artifact Description

This artifact provides the implementation of `AsmetaComp`, a Java-based tool for compositional execution and runtime contract checking of Input/Output Abstract State Machines (I/O ASMs), together with all models and scripts used in the Medicine Reminder and Monitoring (MRM) case study presented in the paper.

The artifact allows reviewers to:

- Execute compositional simulations of I/O ASM components.
- Automatically check assume–guarantee (A/G) contracts at runtime.
- Reproduce the validation scenarios described in Table 1.
- Observe both successful executions and contract violations from the output console.

The artifact is self-contained and executable via a Java Command-Line Interface (CLI) program.

The `AsmetaComp` tool artifact (a Java archive file), all MRM ASMETA models, and composition scripts for validation scenarios reproducibility are available

**Table 1.** Validation Scenarios

| Scenario | Error | Expected violated assertion | Description |
|---|---|---|---|
| CS1 | none | – | A pill is missed, and not rescheduled since it interferes with the next pill. |
| CS2 | none | – | The first pill is taken on time, and the second is rescheduled and taken at the new time. |
| WP | Wrong pill | `inv_A_medicineList` | The patient loads an incorrect pill in the second compartment. |
| PO | Pill overlap | `inv_A_timeConsumption` | The rescheduler sets a new time consumption that violates the minimum interval before the next pill. |

online at: `https://doi.org/10.5281/zenodo.18678488`. A demo video of the tool is also available online at: `https://foselab.unibg.it/asmeta/videos/AsmetaComp@FORTE26.mp4`.

We introduce here four scenarios (see Table 1): two that result in contract violations (*WP* and *PO*) and two that do not (*CS1* and *CS2*). In *CS1*, a pill (e.g., pill "fosamax" in the first compartment) is missed and it is expected that it cannot be rescheduled because the minimum required interval before the next pill (e.g., "moment" in the second compartment) is not satisfied. The second non-violation scenario *CS2* models the situation in which the "fosamax" dose is taken on time, while the "moment" dose is rescheduled and subsequently taken at the new time. Other scenarios are in the online replication package.

### 3.1 Functional outcomes

We here identify all the individual outcomes in the paper, which are reproducible through the artifact. They are the validation scenarios described in Table 1, aimed at showing that our tool is effective in checking assume/guarantee (A/G) contracts during the compositional simulation of I/O ASMs. For each validation scenario, we created a corresponding composition script. Each validation scenario is reproducible, and its outcome can be obtained and verified by running `AsmetaComp` on the corresponding composition script (as better explained in the Quickstart guide below).

*F1:* Scenario *CS1 – A pill is missed, and not rescheduled since it interferes with the next pill* (see Table 1 in the paper) – is expected to produce no contract violations. In this scenario, the pillbox is configured as follows (by the default initialization state `s0` of model `ConfigurationMgr` – see Listing 1 in Appendix B): it has two compartments; the first one contains one pill "fosamax" scheduled at time 360, and the second one contains two pills "moments" scheduled at 730 and 1140. The `minToInterferer` between the two different types of pills is 360.

With this configuration, the scenario is reproduced by the composition script reported in Listing 3. To simulate the patient not taking the "fosamax" pill at time 360, neither in the time interval of 20 minutes, we set the input values in the `run` commands of the composition script appropriately, as shown lines 1–6: the input function `compartmentOpen(1)` is set to false at time 361, 366, and 381 (after 20 minutes from 361).

The `AsmetaComp`'s output by executing the composition script for this scenario is shown in Listing 4. In the `RUN STEP 8` (line 13–17), the fosamax pill

```
1  ...
2  //The user does not open the compartment 1 with "fosamax" pill to be taken at systemTime 360 within 20
       minutes
3  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=361n})
4  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=366n})
5  //20 minutes elapsed and the pill has not been taken since the compartment 1 is still closed, and the system
       does not reschedule it since it interferes with the next pill.
6  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=381n})
7  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=395n})
```

**Listing 3.** Composition script excerpt for scenario CS1

```
1  RUN STEP 5
2  Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=361n ...}
3  Out locations pillbox: {outMess(1)=Take fosamax ...}
4  ...
5  RUN STEP 6
6  Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=366n ...}
7  Out locations pillbox: {outMess(1)=Take fosamax ...}
8  ...
9  RUN STEP 7
10 Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=381n ...}
11 Out locations rescheduler: {setNewTime(compartment1)=false ...}
12 ...
13 RUN STEP 8
14 Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=385n ...}
15 Out locations pillbox:{outMess(1)=fosamax missed, isPillMissed(compartment1)=true, time_consumption(
       compartment1)=[360] ...}
16 Out locations rescheduler: {setOriginalTime(compartment1)=true, setNewTime(compartment1)=true ...}
17 Out locations compartment: {displayMessage(1)=fosamax missed, isPillMissed(compartment1)=true ...}
18 ...
```

**Listing 4.** Output excerpt of the composition script for scenario CS1

is declared missed (the pillbox's out function `isPillMissed(compartment1)` is true), and the rescheduler's out functions `setNewTime(compartment1)` and `setOriginalTime(compartment1)` are true, meaning that the pill has been rescheduled for the next day (indeed, the fosemax consuption time is still 360 as given by the `time_consumption(compartment1)=[360]` in the pillbox's out functions). No contract violations are raised.

*F2:* Scenario *CS2 – the first pill is taken on time, and the second is rescheduled and taken at the new time* – (see Table 1) is expected to produce no contract violations. In this second scenario, the pillbox is configured as in the first scenario *CS1*. Listing 5 reports the script realizing this scenario, and the corresponding `AsmetaComp`'s output is in Listing 6.

To reproduce the desired scenario, by the `run` command we set triggering to our composite system in a way that (*i*) the patient opens the first compartment containing "fosamax" at time 362 (line 3 in Listing 5); (*ii*) The "moment" pill to be taken at time 730 (or within the next 20 minutes) is missed (the input function `compartmentOpen(2)` is kept false till the time instant 796 – see lines

```
1  ...
2  //The user opens the compartment 1 at systemTime 362 with "fosamax" pill to be taken at systemTime 360
3  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=361n})
4  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=true;
       compartmentOpen(2)=false;systemTime=362n})
5  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=364n})
6  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=365n})
7  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=366n})
8  //The user does not open the compartment 2 with "moment" pill to be taken at systemTime 730 within 20
       minutes
9  run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=731n})
10 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=742n})
11 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=762n})
12 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=765n})
13 //The user opens the compartment 2 at systemTime 796 with "moment" pill to be taken at new systemTime
       790
14 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=791n})
15 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=792n})
16 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=793n})
17 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=794n})
18 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=795n})
19 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=true;systemTime=796n})
20 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=797n})
21 run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
       compartmentOpen(2)=false;systemTime=798n})
```

**Listing 5.** Composition script excerpt for scenario CS2

9-12); (*iii*) The "moment" pill is taken at time 796 (line 19), once it has been rescheduled at time 790 by the rescheduler operation.

Evaluating the output of the execution (see Listing 6): (*i*) the pillbox' output prompts that "fosamax" is taken on time (see `RUN STEP 8`: `out_mess` is "fosamax taken", and `actual_timeconsumption(compartment1)` is 365); (*ii*) the "moment" pill is missed and rescheduled at time 790 (see `RUN STEP 12`: `isPillMissed(compartment2)=true` and `newTime(compartment2)= 790`); (*iii*) after several reminders (see `RUN STEP 14--20`: pillbox's `outMess(2)=Take moment`), it is finally taken by the patient (see `RUN STEP 21`: pillbox's `outMess(2) = moment taken` at time 798). No contract violations are raised.

*F3:* Scenario *WP – the patient loads an incorrect pill in the second compartment* – is expected to lead to a contract violation. In this scenario, the pillbox is configured with a wrong pill in the the second compartment ("pantoprazole" instead of "moment"). This configuration is obtained by setting the function `name` of `compartement2` to "pantoprazole" in the default initial state `s0` of the

```
RUN STEP 5
Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=361n ...}
Out locations pillbox: {outMess(1)=Take fosamax ...}...
RUN STEP 6
Running composition: pillbox_comp with input: {compartmentOpen(1)=true, systemTime=362n ...}
Out locations pillbox: {outMess(1)=Take fosamax ...}...
RUN STEP 7
Running composition: pillbox_comp with input: {compartmentOpen(1)=false, systemTime=364n ...}...
RUN STEP 8
Running composition: pillbox_comp with input: {systemTime=365n}
Out locations pillbox:{actual_time_consumption(compartment1)=[365], outMess(1)=fosamax taken ...}...
RUN STEP 9
Running composition: pillbox_comp with input: {systemTime=366n}
Out locations pillbox:{actual_time_consumption(compartment1)=[365], outMess(1)=fosamax taken ...}...
RUN STEP 10
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=371n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 11
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=742n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 12
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=762n}
Out locations pillbox:{isPillMissed(compartment2)=true, newTime(compartment2)=790}
Out locations rescheduler:{newTime(compartment2)=790, setNewTime(compartment2)=true}...
RUN STEP 13
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=765n}
Out locations pillbox:{outMess(2)=moment rescheduled, time_consumption(compartment2)=[790,1140]}...
RUN STEP 14
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=791n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 15
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=792n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 19
Running composition: pillbox_comp with input: {compartmentOpen(2)=true, systemTime=796n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 20
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, systemTime=797n}
Out locations pillbox:{outMess(2)=Take moment}...
RUN STEP 21
Running composition: pillbox_comp with input: {systemTime=798n}
Out locations pillbox:{actual_time_consumption(compartment2)=[798,0], outMess(2)=moment taken}...
```

**Listing 6.** Output excerpt of the composition script for scenario CS2

pillbox model. Other settings are as in the configuration of the previous scenarios. This new faulty model is named `pillboxWrongPill` and is available in the online repository. The scenario is realized by the composition script reported in Listing 7, which sets up the composition with the new faulty model of the pillbox (see the `setup` command at line 7). The tool output for this scenario corresponds to the outcome shown in Listing 8. Once executed, in the `RUN STEP 4`, the tool detects a violation of assumption `inv_A_medicineList` since an incorrect pill was loaded by the user in the second compartment (see the prompted pill names at line 8), which does not match the prescribed ones (see the prompted pill names at line 4).

*F4:* Scenario *PO – the rescheduler sets a new time consumption that violates the minimum interval before the next dose* – is expected to lead to a contract violation. This situation is obtained by changing the rescheduler model

```
//The patient adds the wrong pill to the compartment 2
setup CM as MRMModels/configurationMgr.asm
setup RES as MRMModels/rescheduler.asm
setup setup_comp as ( CM || RES )
setup C1 as MRMModels/compartment.asm
setup C2 as MRMModels/compartment.asm
setup PB as MRMModels/pillboxWrongPill.asm
...
setup pillbox_comp as ( ( PB <|> ( CM || RES ) ) <|> ( C1 || C2 ) )
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=10n;skipNextPill(compartment1)=false;skipNextPill(
    compartment2)=false;skipNextPill(compartment1,compartment2)=false;skipNextPill(compartment2,
    compartment1)=false;newTime(compartment1)=0n;newTime(compartment2)=0n;setOriginalTime(
    compartment1)=false;setOriginalTime(compartment2)=false;setNewTime(compartment1)=false;
    setNewTime(compartment2)=false;openSwitch(1)=false;openSwitch(2)=false;})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=361n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=365n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=366n})
```

**Listing 7.** Composition script excerpt for scenario WP

```
Composition setup for alias: PB -> ASM pillbox: {}
{RES=ASM reschedulerNonOptimal: {}, PB=ASM pillbox: {}, setup_comp= ( ASM configurationMgr: {} ||
    ASM reschedulerNonOptimal: {} ) , CM=ASM configurationMgr: {}, C1=ASM compartment: {}, C2=
    ASM compartment: {}} ...
RUN STEP 1
Running composition: setup_comp with input: {name(compartment2)="moment", name(compartment1)="
    fosamax", ...} ...
RUN STEP 4
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, compartmentOpen(1)=false, ...}
Monitored locations pillbox: {compartmentOpen(2)=false, compartmentOpen(1)=false, ...}
Out locations: pillbox: {name(compartment2)=pantoprazole , name(compartment1)=fosamax, ...}
Monitored locations configurationMgr: {name(compartment1)="fosamax", name(compartment2)=
    "pantoprazole", ...}
Error in composition pillbox_comp: Assumption violation over inv_A_medicineList
Composition script interrupted due to contract violation.
```

**Listing 8.** Composition contract violation: wrong pill

such that the rule `r_rescheduler` for the pill rescheuling omits to consider the `minToInterferer`. This new faulty model is named `reschedulerNonOptimal` and is available in the online repository. Other settings are as in the configuration of the previous scenarios. The scenario is realized by the composition script reported in Listing 9, which sets up the composition with the new faulty model of the rescheduler (see the `setup` command at line 3) and through `run` commands simulates the missing of the "fosamax" pill. The tool outcome is shown in Listing 10 of the paper. The tool detects a violation of assumption `inv_A_timeConsumption` shown in the Appendix B in Listing 1 of the paper. The rescheduler sets 420 as new time consumption for the missed "fosamax" pill (see line 7 of Listing 10), which violates the minimum time interval (`minToInterferer = 360`) before the assumption of "moment" at time 730 (the time difference is 310 that is less than 360).

```
//Fosamax is missed and it is not possible to reschedule it otherwise the minToInterfer is not satisfied
setup CM as MRMModels/configurationMgr.asm
setup RES as MRMModels/reschedulerNonOptimal.asm
setup setup_comp as ( CM || RES )
setup C1 as MRMModels/compartment.asm
setup C2 as MRMModels/compartment.asm
setup PB as MRMModels/pillbox.asm
...
setup pillbox_comp as ( ( PB <|> ( CM || RES ) ) <|> ( C1 || C2 ) )
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=10n;skipNextPill(compartment1)=false;skipNextPill(
    compartment2)=false;skipNextPill(compartment1,compartment2)=false;skipNextPill(compartment2,
    compartment1)=false;newTime(compartment1)=0n;newTime(compartment2)=0n;setOriginalTime(
    compartment1)=false;setOriginalTime(compartment2)=false;setNewTime(compartment1)=false;
    setNewTime(compartment2)=false;openSwitch(1)=false;openSwitch(2)=false;})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=361n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=365n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=366n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=731n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=742n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=753n})
run(pillbox_comp, {myID=undef;outMess(undef)="";led(undef)=OFF;compartmentOpen(1)=false;
    compartmentOpen(2)=false;systemTime=754n})
```

**Listing 9.** Composition script excerpt for scenario PO

Additional functional outcomes corresponding to two additional validation scenarios are described in the online replication repository (see `Readme.pdf` file).

## 4 Quickstart

This is a quick start guide explaining how the artifact environment is set up and how run `AsmetaComp` on the composition scripts for the validation scenarios.

The artifact can be executed on any platform that has a working Java environment. If Java is not installed, follow the guidelines in Section C.2.2 and then continue with Section C.2.1.

A high-level directory structure of the submitted tool artifact is shown in Table 2.

### 4.1 Executing `AsmetaComp` to reproduce the functional outcomes

- Download the replication package from Zenodo `https://doi.org/10.5281/zenodo.18678488`
- Extract the content of the `AsmetaCompScript.zip`.
- Open the Command Prompt/Terminal in the extracted folder and run the following command to run `AsmetaComp` on a composition script:
  `java -jar AsmetaComp.jar MRMModels/"filename".asmsh`
  where `"filename"` is the composition script to execute.

The specific commands for the four validation scenarios of the paper are as follows:

```
Composition setup for alias: PB —> ASM pillbox: {}
{RES=ASM reschedulerNonOptimal: {}, PB=ASM pillbox: {}, setup_comp= ( ASM configurationMgr: {} ||
    ASM reschedulerNonOptimal: {} ) , CM=ASM configurationMgr: {}, C1=ASM compartment: {}, C2=
    ASM compartment: {}} ...
RUN STEP 10
Running composition: pillbox_comp with input: {compartmentOpen(2)=false, compartmentOpen(1)=false,
    systemTime=753n}
Monitored locations pillbox: {systemTime=753n, openSwitch(1)=false, openSwitch(2)=false,
    setNewTime(compartment1)=true, displayMessage(1)="fosamax missed", displayMessage(2)="Take
    moment in 10 minutes"}
Out locations pillbox: {led(1)=OFF, led(2)=OFF, outMess(1)=fosamax rescheduled, outMess(2)=moment
    missed, redLed(compartment1)=OFF, redLed(compartment2)=OFF,
    time_consumption(compartment1)=[420], time_consumption(compartment2)=[730,1140], name(
    compartment1)=fosamax, name(compartment2)=moment, pillboxSystemTime=753}
Monitored locations configurationMgr: {minToInterferer(fosamax,moment)=360,
    time_consumption(compartment1)=[420], time_consumption(compartment2)=[730,1140],... }
Error in composition pillbox_comp: Assumption violation over inv_A_timeConsumption
Composition script interrupted due to contract violation.
```

**Listing 10.** Composition contract violation: time overlap with the next medicine

**F1:** `java -jar AsmetaComp.jar MRMModels/pillboxCompCS1.asmsh`
**F2:** `java -jar AsmetaComp.jar MRMModels/pillboxCompCS2.asmsh`
**F3:** `java -jar AsmetaComp.jar MRMModels/pillboxCompWP.asmsh`
**F4:** `java -jar AsmetaComp.jar MRMModels/pillboxCompPO.asmsh`

### 4.2 Installing Java

***Windows*** To quickly install Java on Mac, follow these three steps:

**1. Download the Installer:**

Download the latest JDK (Java Development Kit) installer (MSI or .exe) from the Oracle website.

**2. Run the installer:**

Double-click the downloaded .msi or .exe file to start the installation wizard. Follow the on-screen prompts, and click "Close" when finished.

**3. Verify the installation (optional):**

Open Command Prompt (search for cmd in the Start menu) and type `java -version` to confirm that the installation was successful.

***MacOS*** To quickly install Java on Mac, follow these three steps:

**Download the Installer:**

Visit the official Oracle website and download the latest JDK (Java Development Kit) DMG installer. Choose the version based on your processor:

*ARM64 DMG Installer:* For Mac with Apple Silicon (M1/M2/M3 chips).

*x64 DMG Installer:* For Intel-based Macs.

**Run the Installer:**

Open the downloaded .dmg file and double-click the .pkg file to launch the installation wizard. Follow the on-screen instructions, click "Install," and enter your Mac's password to grant permission.

**Verify Installation (optional):**

Open the Terminal (via Spotlight search), type java -version, and press Enter to confirm the installation.

| File name | Description |
|---|---|
| Readme.pdf | Readme file |
| How to install Java.pdf | Instruction to install Java |
| AsmetaCompScript.zip | |
| MRM_CBD/ | Folder with AsmetaComp.jar and functionality outcome examples |
| AsmetaComp.jar | AsmetaComp .jar file |
| MRMModels/ | Folder with models and scripts |
| compartment.asm | Asmeta model of Compartment component |
| configurationMgr.asm | Asmeta model of the ConfigurationMgr component |
| knowledge.asm | Asmeta module imported in Pillbox and Rescheduler components |
| pillbox.asm | Asmeta model of the Pillbox component |
| pillboxCompCS1.asmsh | Script of scenario CS1 |
| pillboxCompCS2.asmsh | Script of scenario CS2 |
| pillboxCompCS3.asmsh | Script of scenario CS3 |
| pillboxCompPO.asmsh | Script of scenario PO |
| pillboxCompWC.asmsh | Script of scenario WC |
| pillboxCompWP.asmsh | Script of scenario WP |
| pillboxWrongConfig.asm | Asmeta model of the wrong Pillbox component used in scenario WC |
| pillboxWrongPill.asm | Asmeta model of the wrong Pillbox component used in scenario WP |
| rescheduler.asm | Asmeta model of the Rescheduler component |
| reschedulerNonOptimal.asm | Asmeta model of the wrong Rescheduler component used in scenario PO |
| reschedulerWrongConfig.asm | Asmeta model of the wrong Rescheduler component |

Table 2. High-level directory structure of the submitted tool artifact

***Linux - Ubuntu*** To quickly install Java on Ubuntu, follow these three steps:

First verify that Java is not already inststalled Install the default JRE with apt Run `java --version` on the command line to validate the install.

**Check for existing Java installs:**

You don't need to setup Java on Ubuntu if someone has already installed it for you. A quick `java --version` check will tell you if an install is necessary. Open a terminal and check if java is not installed, by trying to execute java:

```
java -version
Command 'java' not found, but can be installed with:
sudo apt install default-jre
```

If instead, the command indicates the version, then Java is installed and you are done.

**Install Java on ubuntu with apt:**

The full Java suite includes the Java Runtime Environment (JRE), Java Virtual Machine (JVM) and utilities to develop Java source code. The Java Development Kit (JDK) provides everything a user needs to run Java applications. Install the JDK with the following command:

```
sudo apt install default-jre
Setting up default-jdk-headless (2:1.21) ...
Setting up openjdk-21-jdk:amd64 (21.0.5+11-1ubuntu 24.04.1) ...
using /usr/lib/jvm/java-11-openjdk-amd64/bin/jconsole
Setting up default-jdk (2:1.21) ...
```

Note that it may be necessary to execute sudo apt-get update

**Verify Java is on the PATH:**

To validate the successful installation of Java on Ubuntu, and to ensure the JDK's bin directory is accessible through the OS's PATH variable, issue the following command:

```
java -version
openjdk version "21.0.5" 2024-10-15
OpenJDK Runtime Environment (21.0.5+11-1ubuntu 24.04.11)
OpenJDK 64-Bit Server VM (21.0.5+11-1ubuntu 24.04.1, mixed mode,
sharing)
```

If the `java -version` command outputs information about the version of Java installed, along with information about the JVM and runtime environment, then you successfully installed Java on Ubuntu with apt.

## 5 Conclusion

This artifact presented AsmetaComp, a tool for Contract-based Design and runtime contract checking of I/O ASM models. The paper to which this artifact refers has been accepted at FORTE 2026 - 46th International Conference on Formal Techniques for Distributed Objects, Components, and Systems.

## References

1. Bonfanti, S., Gargantini, A., Riccobene, E., Scandurra, P.: A compositional simulation framework for Abstract State Machine models of Discrete Event Systems. Formal Aspects of Computing (2024)
2. Bonfanti, S., Riccobene, E., Scandurra, P.: A runtime safety enforcement approach by monitoring and adaptation. In: Biffl, S., Navarro, E., Löwe, W., Sirjani, M., Mirandola, R., Weyns, D. (eds.) Software Architecture. pp. 20–36. Springer International Publishing, Cham (2021)